ECE551: Homework 3

Due: Monday, October 12th @ 11:55PM

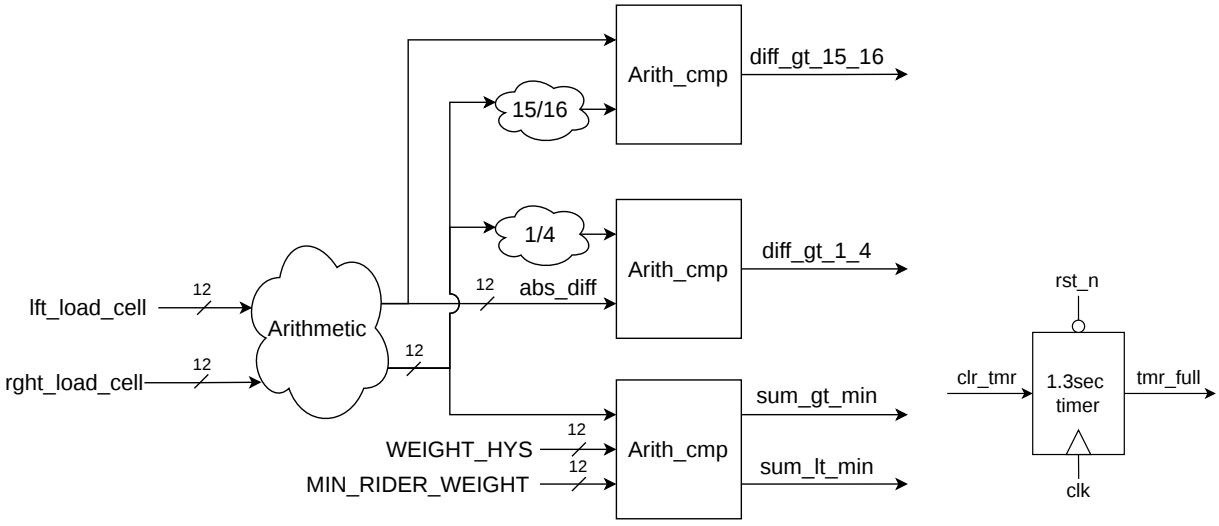
For this homework, you *must*:

- Work individually.
- Use descriptive signal names and comment your code
- Remember what you learned from the Cummings SNUG paper

1 (20pts) SM Design

Stepping onto a Segway is a bit disconcerting and we want the rider to be comfortably on and balanced before enabling steering. There are two load sensors in the floor of the Segway that measure force each foot is exerting on the floor. We will use measurements from these load sensors to determine when a rider is on, and reasonably balanced side to side. Your first task in this problem will be to draw a bubble diagram for the steering enable state machine (*steer_en_SM*).

Assume the following hardware elements are available to you:



(a) Datapath (b) Timer

Figure 1: Hardware elements available. The timer always counts up.

Desired Behavior:

First we look to see that the sum of the two load cells exceeds the minimum rider weight. If that condition is met (***sum_gt_min***), we exit the initial state and enter a state looking for steady (>1.3sec) side to side balance. If **diff_gt_1_4**, the rider is not balanced (side to side) and we clear the 1.3 sec timer (**clr_tmr**) and remain in this state. If **diff_gt_1_4** has been low for 1.3 seconds, then we know the rider is balanced and we enter the steering enabled state. We can leave the steering enabled state one of two ways: 1) the rider suddenly gets knocked off the device (**sum_lt_min** goes high), or 2) the difference between the load cells exceeds 15/16 of the sum (rider is stepping off). Under both conditions we exit the steering enabled state.

If the user is stepping off, we return to the state for waiting for side to side balance for 1.3 sec. In this state there is a transition back to the initial state if **sum_lt_min** rises (the rider completely stepped off). If the user is knocked off, we immediately return to the initial state from the steering enabled state. The check for "knocked off" should have priority over the check for stepping off.

Your state machines should output 3 signals:

- **clr_tmr**: Clears the 1.3 sec timer.
- **en_steer**: Enables steering of the Segway. Should only be asserted when the rider is on the Segway and has been balanced (side to side) for at least 1.3 sec.
- **rider_off**: Indicates when a rider is no longer on the Segway. Should be asserted whenever the rider weight is not known to exceed the minimum threshold (i.e., when in the initial state or when in another state and **sum_lt_min**).

  This signal goes to the authorization block responsible for turning Segway on/off. It is used to prevent the power off of the Segway while a rider is on. (Imagine you are riding your friend's Segway and they took out their phone and commanded it to power off. Since it actively balances the rider, we cannot allow power off to occur unless we know the rider is off).

Draw the bubble diagram for this state machine. If any outputs have an assumed default value, clearly state it in your diagram. Take a picture with your phone and submit it. It must be legible!

You will now implement that SM in Verilog and testing it via a provided testbench.

- Download **steer_en_SM_shell.sv** and **steer_en_SM_tb.sv**.
- Flush out **steer_en_SM_shell.sv** and rename it **steer_en_SM.sv**.
- Use the provided testbench (**steer_en_SM_tb.sv**) to test it in ModelSim. Debug any errors the testbench points out.

Your state machine should have the following interface:

Signal	Direction
clk, rst_n	in
tmr_full	in
sum_gt_min	in
sum_lt_min	in
diff_gt_1_4	in
diff_gt_15_16	in
clr_tmr	out
en_steer	out
rider_off	out

Submit: a picture of your **bubble diagram**, **steer_en_SM.sv**, and **proof** that the testbench ran to completion on your implementation.

2 (15pts) PWM11

See Exercise 09 specification. Did you complete it well? If not, polish it up.

Submit: PWM11.sv, PWM11_tb.sv, and **proof** (waveforms) that it ran correctly with different duty cycles.

3 (20pts) PID

See Exercise 10 specification. Did you complete it well? If not, polish it up.

Submit: PID.sv and proof that it passed the provided self-checking testbench. Make sure your username can be clearly seen on the image.

4 (15pts) UART Transmitter

See Exercise 11 specification. Did you complete it well? If not, polish it up.

Submit: UART_tx.sv

5 (30pts) UART Receiver and Test

See Exercise 12 specification. Did you complete it well? If not, polish it up.

Submit: UART_rx.sv, **UART_tb.sv**, and **proof** that your self-checking testbench passed. Make sure your username can be clearly seen in the images.